

6.1 Introduction

This document presents the architecture and detailed design for the Phishing Detection Chrome Extension project. The system is a browser-based cybersecurity tool that analyzes visited web pages in real time to detect phishing indicators and warn users before they submit sensitive information.

The extension integrates client-side analysis, heuristic rule evaluation, optional machine learning classification, and a lightweight backend service for threat intelligence updates.

6.1.1 System Objectives

The objective of this application is to:

- Detect phishing websites in real time within the Chrome browser.
- Alert users before credential submission occurs.
- Analyze URL structure, domain age, SSL certificate validity, and page content features.
- Provide a risk score and explanation for transparency.
- Maintain low latency and minimal performance impact.
- Optionally store flagged URLs in a database for analytics and model improvement.

The system aims to reduce successful phishing attacks by providing proactive, explainable detection directly within the browser environment.

6.1.2 Hardware, Software, and Human Interfaces

6.1.2.1 Hardware Interfaces

- User device (Windows/macOS/Linux laptop or desktop)
- Internet connection
- Standard input devices (mouse, keyboard)

6.1.2.2 Software Interfaces

- **Google Chrome Browser (Manifest V3)**
- Chrome Extension APIs:
 - `chrome.webRequest`
 - `chrome.tabs`

- `chrome.storage`
- `chrome.runtime`
- HTTPS protocol for backend communication
- Optional REST API server (Node.js or Python Flask)
- Optional database (MongoDB / PostgreSQL)

6.1.2.3 Third-Party Libraries

- URL parsing library (e.g., `validator.js`)
- Machine learning model (TensorFlow.js or pre-trained lightweight model)
- WHOIS lookup API (if used)

6.1.2.4 Human Interface (GUI)

The user interface consists of:

- Extension popup window
- Risk score display (Low / Medium / High)
- Explanation panel listing detected risk features
- Warning overlay on suspicious pages

Example interface behavior:

- Green indicator → Safe
 - Yellow indicator → Suspicious
 - Red overlay → High phishing probability
-

6.2 Architectural Design

The system follows a **modular client-server architecture** with browser-based primary processing.

6.2.1 Major Software Components

1. **Content Script Module**
 - Injected into web pages
 - Extracts DOM features
 - Detects form fields and suspicious inputs
2. **Background Service Worker**
 - Manages URL analysis
 - Handles API communication
 - Coordinates extension logic

3. **Phishing Detection Engine**
 - Heuristic analyzer
 - Feature extractor
 - Risk scoring algorithm
 - Optional ML classifier
 4. **Popup UI Module**
 - Displays risk score
 - Shows detected warning indicators
 5. **Backend Threat Intelligence Server (Optional)**
 - Stores flagged URLs
 - Provides blacklist updates
 - Supports analytics
 6. **Database (Optional)**
 - Stores phishing reports
 - Tracks detection statistics
-

6.2.2 Major Software Interactions

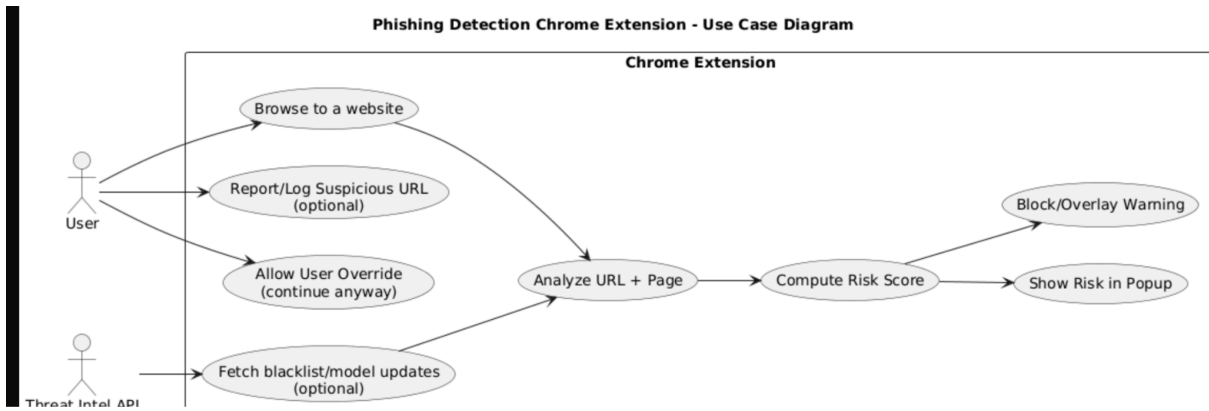
1. User visits website
2. Content script extracts features
3. Background service worker receives URL + feature data
4. Detection engine evaluates:
 - URL structure
 - Domain length
 - Presence of IP address in URL
 - SSL validity
 - Form action mismatch
5. Risk score generated
6. UI displays result
7. If high risk:
 - Warning overlay injected
 - Optional backend log entry

Communication occurs through:

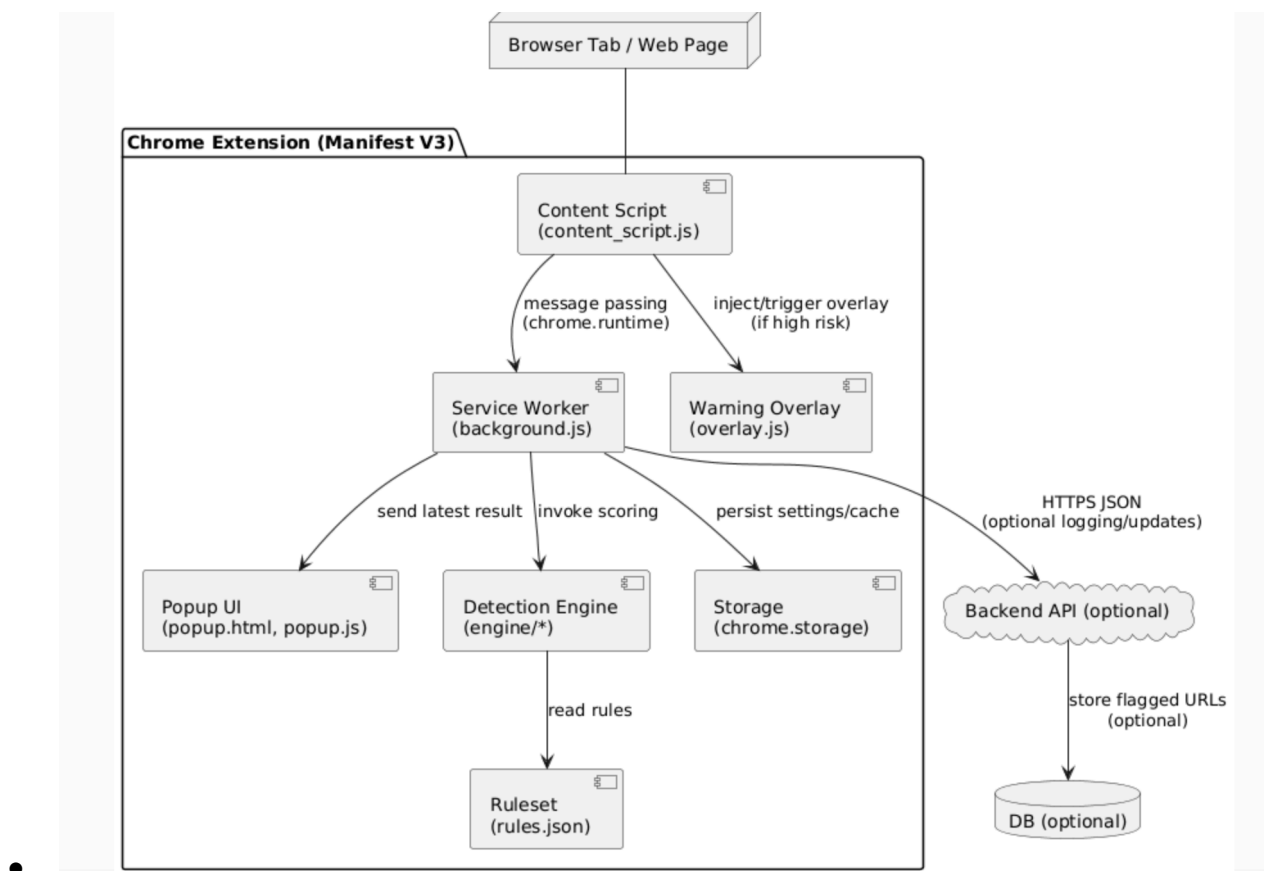
- Chrome message passing (`chrome.runtime.sendMessage`)
 - HTTPS REST API calls (JSON format)
-

6.2.3 Architectural Design Diagrams

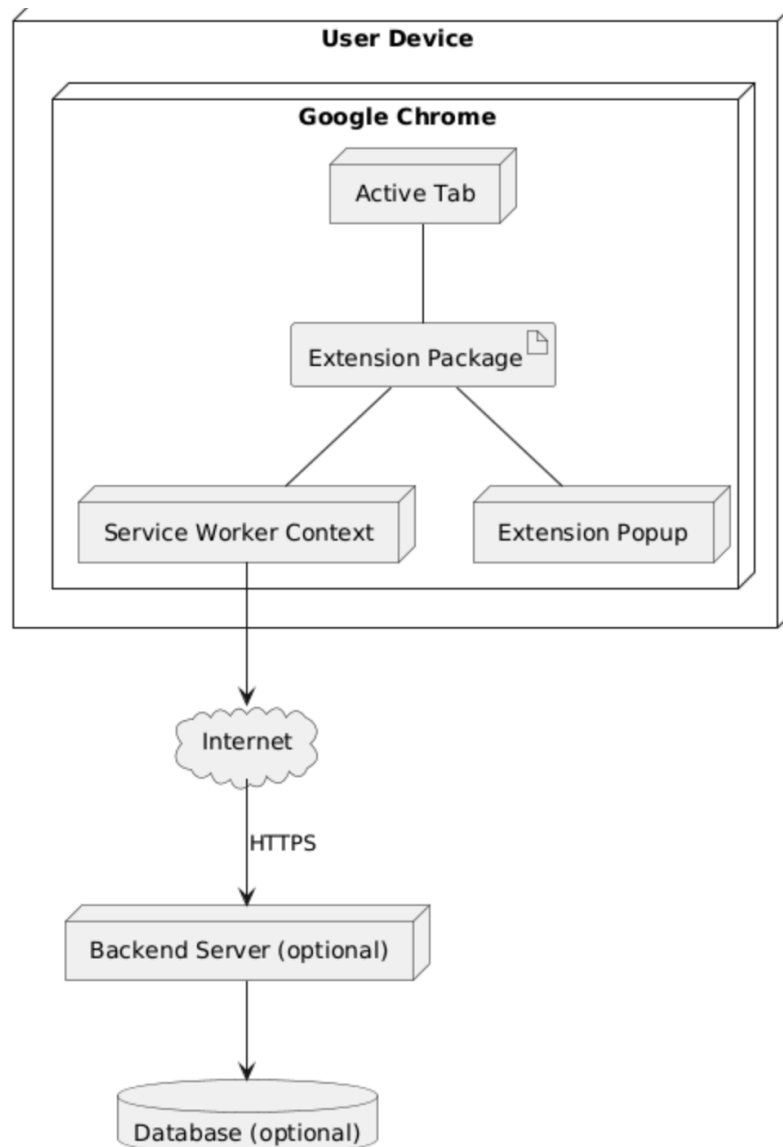
1. Use Case Diagram



2. Component Diagram



3. Deployment Diagram



6.3 CSC and CSU Descriptions

CSCI: Phishguard

CSC 1: Browser Interaction Layer

- ContentScript.js
- Background.js

CSC 2: Detection Engine Layer

- FeatureExtractor.js
- HeuristicAnalyzer.js
- MLClassifier.js

CSC 3: User Interface Layer

- Popup.html
- Popup.js
- WarningOverlay.js

CSC 4: Backend System (Optional)

- Server.js
 - DatabaseConnector.js
-

6.3.1 Detailed Class Descriptions

6.3.1.1 FeatureExtractor

Purpose: Extract phishing-related features from URL and DOM.

Fields:

- url
- domain
- pageContent
- featureVector

Methods:

- extractURLFeatures()
 - extractDOMFeatures()
 - getFeatureVector()
-

6.3.1.2 HeuristicAnalyzer

Purpose: Apply rule-based phishing detection.

Fields:

- suspiciousPatterns
- riskScore

Methods:

- checkIPInURL()
- checkLongURL()
- checkSSL()

- `computeRiskScore()`
-

6.3.1.3 MLClassifier (Optional)

Purpose: Predict phishing probability using trained model.

Fields:

- `model`
- `predictionScore`

Methods:

- `loadModel()`
 - `predict(featureVector)`
-

6.3.1.4 WarningOverlay

Purpose: Inject visual warning into page.

Fields:

- `warningMessage`
- `severityLevel`

Methods:

- `displayWarning()`
 - `removeWarning()`
-

6.3.2 Detailed Interface Descriptions

Browser ↔ Content Script

- DOM extraction
- Form submission monitoring

Content Script ↔ Background Worker

- JSON message passing

Background ↔ Backend API

- HTTPS POST requests
 - JSON payload
-

6.3.3 Detailed Data Structure Descriptions

Feature Vector Structure

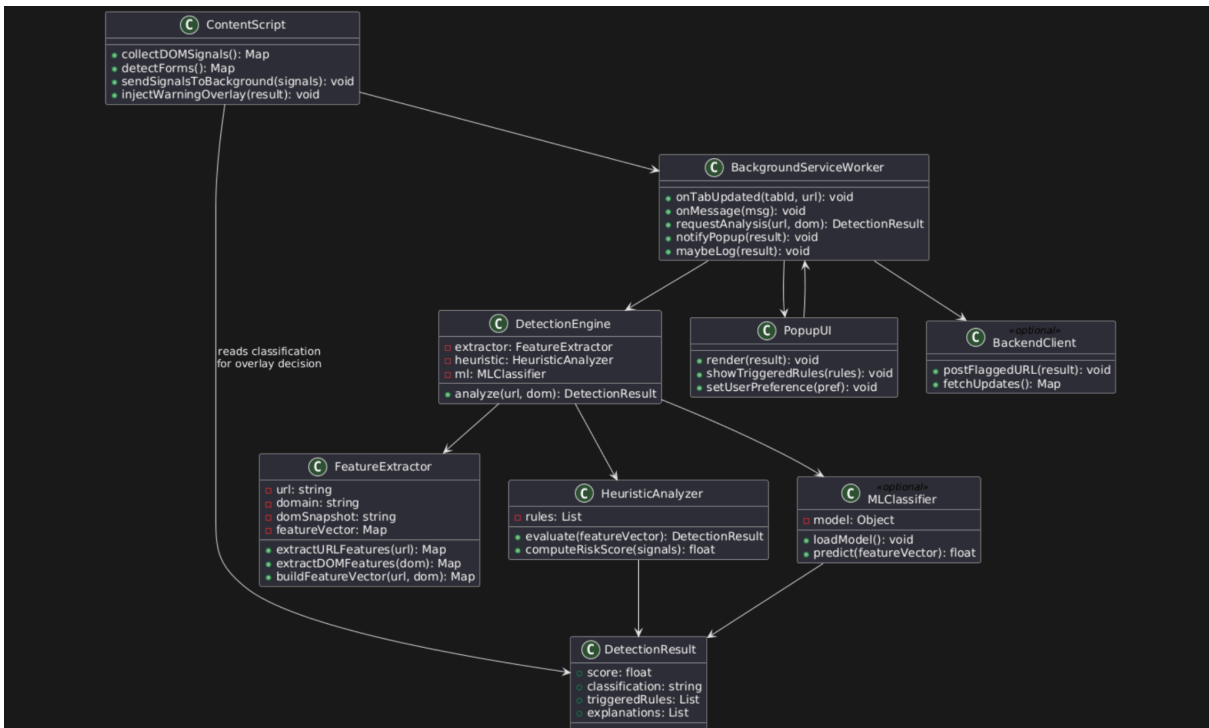
```
{  
  url_length: number,  
  contains_ip: boolean,  
  num_subdomains: number,  
  ssl_valid: boolean,  
  form_action_mismatch: boolean,  
  domain_age_days: number  
}
```

Risk Score Object

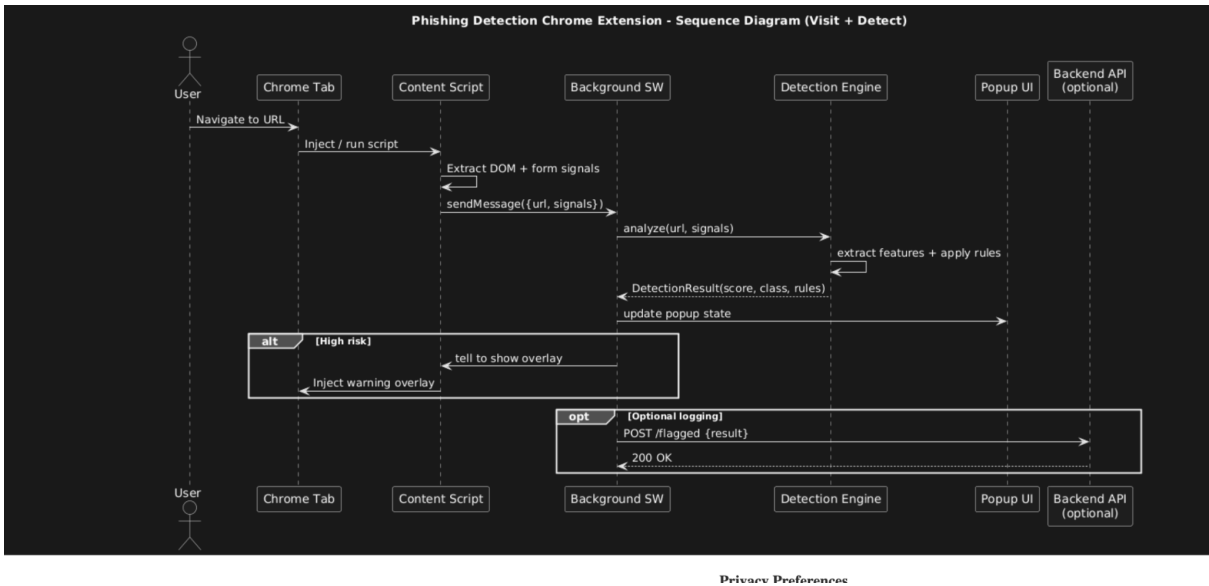
```
{  
  score: float,  
  classification: "Safe | Suspicious | Phishing",  
  triggered_rules: [string]  
}
```

6.3.4 Detailed Design Diagrams

Top-Level Class Diagram



Sequence Diagram



Activity Diagram



6.4.1 ER Diagram

This project does not utilize a database. All phishing detection and risk scoring operations are performed locally within the Chrome extension environment. No persistent server-side storage is required for system functionality.

6.4.2 Database Access

- Accessed via REST API
- JSON request/response
- Authentication via API key
- Secure HTTPS connection

6.4.3 Database Security

- Encrypted HTTPS communication
- Input validation to prevent injection
- Rate limiting
- Secure server hosting
- Sanitized URL storage